

Findin

A WHERE and WHEN guessing game for living and extinct animals, built around reproducible daily puzzles and cautious scientific content handling.

Role: builder (ownership pending confirmation)

Private full-stack product

Next.js

TypeScript

Supabase



80

published/playable content rows

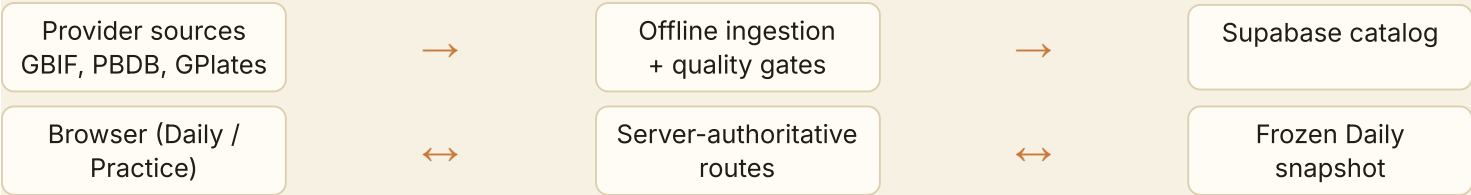
578

total content candidates staged

328

passing tests, 53 test files

SYSTEM SUMMARY



KEY ENGINEERING DECISIONS

- Persist a Daily puzzle row with a frozen animal snapshot, so a player never sees the answer move underneath them.
- Run ingestion, rescue, and validation offline so gameplay never depends on live provider calls.
- Cache GPlates-derived coastlines and fossil coordinates ahead of time, with fallback to the modern map for unsupported ages.
- Apply a hard no-repeat Daily rotation before scoring the remaining candidates for balance.

DATA + QUALITY

- Provider evidence is normalized and gated through explicit content states before it can become playable.
- Quality gates cover taxonomy, WHERE, WHEN, paleo support, media, and Field Notes.

LIMITATIONS

- No live demo link; the previously documented Railway URL returned 404.
- Of 578 staged candidates, 80 are published/playable; the Daily-ready pool is smaller still.
- The globe view is experimental; the map/timeline flow is the production-safe claim.

WHAT IT DEMONSTRATES

Turning messy scientific data into something that has to be right — offline ingestion and validation, server-authoritative game state, and being explicit about which surfaces are production-safe versus experimental.

Public-safe case study. No app source code, credentials, private repository links, account identifiers, or internal deployment identifiers included.